

1. Vue d'ensemble du système de rôles

1.1 Rôles définis

6 rôles sont définis dans l'enum `UserRole` (`backend/app/models.py:41-47`) :

#	Valeur technique	Nom fonctionnel	Portée
1	<code>student</code>	Élève	École (school_id obligatoire)
2	<code>teacher</code>	Enseignant	École (school_id obligatoire)
3	<code>admin_school</code>	Administrateur d'école	École (school_id obligatoire)
4	<code>super_admin</code>	Super Administrateur plateforme	Toutes les écoles
5	<code>pedagogical_admin</code>	Responsable pédagogique plateforme	Toutes les écoles
6	<code>pedagogical_lead</code>	Responsable pédagogique école	Une école (school_id obligatoire)

Rôle implicite non documenté : `ClassroomEnrollment.role` (`models.py:659`) admet les valeurs `"student"` ou `"monitor"` — un élève peut être "monitor" dans une classe sans que cela affecte ses permissions globales.

1.2 Hiérarchie des rôles

`super_admin`

- `admin_school` (par école)
- `pedagogical_admin` (plateforme)
- `pedagogical_lead` (par école, school_id requis)
 - `teacher` (par école)
 - `student` (par école)

Clé de dépendance RBAC (`deps.py:22-76`) :

- `require_admin` → `super_admin`, `admin_school`, `pedagogical_admin`, `pedagogical_lead`

- `require_super_admin` → `super_admin` uniquement
- `require_teacher_or_admin` → `super_admin`, `admin_school`, `teacher`
- `require_pedagogical_admin` → `pedagogical_admin` uniquement
- `require_pedagogical_lead` → `pedagogical_lead` uniquement (+ `school_id` requis)
- `require_pedagogical_any` → `pedagogical_admin`, `pedagogical_lead`

1.3 Sous-catégories d'enseignants

Le champ `SubscriptionPlan` (`models.py:56-60`) distingue 3 états pour les enseignants :

Plan	Valeur	Description
<code>TRIAL</code>	<code>"trial"</code>	Essai gratuit, lié à l'école de démonstration
<code>SCHOOL_AFFILIATED</code>	<code>"school_affiliated"</code>	Approuvé par <code>admin_school</code>
<code>INDEPENDENT_PAID</code>	<code>"independent_paid"</code>	Enseignant indépendant payant

2. Fiche détaillée par rôle

2.1 `student` — Élève

Nom technique : `"student"` (`models.py:42`) Description : Utilisateur final qui consomme du contenu éducatif.

Fonctionnalités autorisées :

Fonctionnalité	Type	Endpoints
Parcourir le catalogue public	Lecture	<code>GET /catalog/courses</code> (public), <code>GET /api/learner/courses</code> (public)
S'inscrire à un cours	Écriture	<code>POST /api/learner/courses/{id}/enroll</code>
Consulter ses cours	Lecture	<code>GET /api/learner/my-courses</code>
Suivre une leçon	Lecture	<code>GET /api/learner/lessons/{id}</code>
Marquer progression	Écriture	<code>POST /api/learner/lessons/{id}/progress</code>
Passer un quiz	Écriture	<code>POST /api/learner/quizzes/{id}/start</code> , <code>POST .../submit</code>
Consulter certificats	Lecture	<code>GET /api/learner/certificates</code>

Fonctionnalité	Type	Endpoints
Ajouter notes/-bookmarks	Écriture	<code>POST /api/learner/lessons/{id}/notes</code> , <code>.../bookmarks</code>
Utiliser le tuteur IA	Écriture	<code>POST /api/ai/ask</code> , <code>POST /api/ai/explain</code> , <code>POST /api/ai/quiz</code>
Acheter un pack	Écriture	<code>POST /api/packs/{id}/purchase</code>
Consulter portefeuille	Lecture	<code>GET /api/wallet/balance</code> , <code>GET /api/wallet/history</code>
Acheter des crédits	Écriture	<code>POST /api/wallet/purchase</code>
Consulter boîte réception	Lecture	<code>GET /api/inbox/messages</code>
Soumettre vérification identité	Écriture	<code>POST /api/learner/verify-identity</code>

Fonctionnalités interdites :

- Créer/modifier/supprimer des cours
- Gérer des classes ou des élèves
- Accéder au panneau admin
- Gérer les settings plateforme
- Approuver des cours pour B2B

Endpoints API accessibles : 22 endpoints

(`learner.py`, `wallet.py`, `inbox.py`, `ai.py`, `packs.py`) Pages frontend
`/dashboard` (StudentDashboard), `courses`, `courses/:id`, `assignments`, `ai-tutor`, `wallet`, `inbox`

2.2 `teacher` — Enseignant

Nom technique : `"teacher"` (`models.py:43`) Description : Crée du contenu éducatif et gère ses classes.

Fonctionnalités autorisées :

Nom technique :	"admin_school" (models.py:44)	Description :	Gère une école spécifique : utilisateurs, cours, finances, settings.
Fonctionnalités autorisées :			

Fonctionnalité	Type	Endpoints
Tout ce qu'un teacher peut faire	*	(voir 2.2)
Gérer les utilisateurs de son école	CRUD	<code>GET/POST/PUT/DELETE /api/admin/users*</code>
Créer/modifier des classes	CRUD	<code>POST /api/admin/classes</code> , <code>POST /api/admin/enrollments</code>
Gérer les cours de son école	CRUD	Tous les endpoints <code>admin_courses.py</code>
Approuver/rejeter enseignants	Écriture	<code>POST /api/admin/teacher-registrations/{id}/review</code>
Aperçu analytics école	Lecture	<code>GET /api/admin/analytics/overview</code> , <code>.../users</code>
Gérer settings école	Écriture	<code>PUT /api/admin/settings</code> (portée école)
Envoyer broadcast	Écriture	<code>POST /api/admin/broadcast</code>
Achat pack école	Écriture	<code>POST /api/admin/school/packs/purchase</code>
Import CSV élèves	Écriture	<code>POST /api/admin/import-students/upload</code>
Consulter portefeuille école	Lecture	<code>GET /api/admin/wallets</code>
Gérer transactions école	CRUD	<code>POST /api/admin/transactions</code>
Voir dashboard école	Lecture	<code>GET /api/admin/dashboard</code>
Gérer token packages	CRUD	<code>POST /api/admin/token-packages</code>

Fonctionnalités interdites (réservées super_admin) :	
	• Supprimer un utilisateur (<code>DELETE /api/admin/users/{id}</code>) — super_admin only) • Supprimer une école (<code>DELETE /api/admin/schools/{id}</code>) — super_admin only) • Modifier les settings plateforme (clés API, providers IA) • Voir les stats globales (<code>GET /api/admin/stats/global</code>) • Voir les audit logs de toutes les écoles • Gérer les abonnements expirés • Allouer des crédits à un wallet (super_admin only)

- Voir le rapport de marge

Endpoints API accessibles : ~65 endpoints

([admin.py](#), [admin_courses.py](#), [admin_chapters.py](#), [admin_lessons.py](#), [admin_quizzes.py](#), [media.py](#), [logs.py](#)) Pages frontend : [/dashboard/school](#) (SchoolAdminLayout) — overview, users, finance, courses, teachers, settings, pedagogical

2.4 [super_admin](#) — Super Administrateur

Nom technique : ["super_admin"](#) ([models.py:45](#)) Description : Accès total à toute la plateforme, toutes les écoles.

Fonctionnalités autorisées : TOUT (superset de tous les autres rôles)

Privilèges exclusifs :

Fonctionnalité	Endpoints
Supprimer un utilisateur	DELETE /api/admin/users/{id} (admin.py:1467)
Supprimer une école	DELETE /api/admin/schools/{id} (admin.py:1975)
Voir stats globales	GET /api/admin/stats/global (admin.py:1382)
Voir toutes les transactions	GET /api/admin/transactions (admin.py:1488)
Gérer vérifications identité	GET /api/admin/verifications/pending (admin.py:1921)
Vérifier un utilisateur	POST /api/admin/verifications/{id}/review (admin.py:1942)
Voir abonnements expirés	GET /api/admin/subscriptions/expiring (admin.py:2067)
Déclencher vérification expirations	POST /api/admin/subscriptions/check-expirations (admin.py:2093)
Étendre un abonnement	POST /api/admin/users/{id}/extend-subscription (admin.py:2103)
Rapport de marge	GET /api/admin/wallet/margin-report (admin.py:2220)
Accorder/révoquer cours B2B	POST /api/admin/courses/schools/{school_id}/grant-course (admin_courses.py:839)

Fonctionnalité	Endpoints
Accéder à tous les audit logs	<code>GET /api/admin/audit-logs</code> (admin.py:2022)
Allouer des crédits wallet	<code>POST /api/admin/wallet/allocate</code> (admin.py:2133)
Rapport consommation	<code>GET /api/admin/wallet/consumption-report</code> (admin.py:2173)
Approuver cours pour B2B	<code>PUT /api/pedagogical/courses/{id}/review</code> (pedagogical.py:92)

Endpoints API accessibles : Tous (~210 endpoints) Pages frontend
: `/dashboard/admin` (AdminLayout) — dashboard, users, schools, courses, analytics, teachers, finance, media, inbox, ai-factory, course-distribution, teacher-catalog, broadcast, settings, packages, audit, pedagogical-review

2.5 `pedagogical_admin` — Responsable pédagogique plateforme
Nom technique : `"pedagogical_admin"` (models.py:46) Description : Supervise la qualité pédagogique des contenus sur toutes les écoles. Approuve les cours pour distribution B2B.
Fonctionnalités autorisées :

Fonctionnalité	Type	Endpoints
Voir cours en attente	Lecture	<code>GET /api/pedagogical/courses/pending</code> (pedagogical.py:54)
Approuver cours pour B2B	Écriture	<code>PUT /api/pedagogical/courses/{id}/review</code> (pedagogical.py:92)
Demander révision	Écriture	(même endpoint, action <code>"needs_revision"</code>)
Voir couverture curriculum	Lecture	<code>GET /api/pedagogical/reports/curriculum-coverage</code> (pedagogical.py:119)
Gérer escalades	CRUD	<code>GET/PUT /api/pedagogical/escalations*</code> (pedagogical.py:233, 264)
Gérer rapports IA	CRUD	<code>GET/PUT /api/pedagogical/reports*</code> (pedagogical.py:178, 209)
CRUD packs d'étude	CRUD	<code>POST/PUT /api/pedagogical/packs*</code> (pedagogical.py:287-379)

Fonctionnalité	Type	Endpoints
Tout ce qu'un admin_school fait	*	(via <code>require_admin</code>)
Archiver un cours	Écriture	Via lifecycle service
Accéder à AdminCoursesPage	UI	<code>/dashboard/admin/courses</code>
Accéder à PedagogicalAdminPage	UI	<code>/dashboard/admin/pedagogical-review</code>

Distinction clé avec `pedagogical_lead` : Seul `pedagogical_admin` peut approuver pour B2B (`approved_for_b2b`). Le `pedagogical_lead` ne peut approuver que localement (`approved_local`) ou escalader.

Endpoints API accessibles : 13 endpoints pédagogiques + ~65 endpoints admin

(via `require_admin`) Pages frontend : `/dashboard/admin/*` (AdminLayout) avec accès exclusif à "Review Pedago."

2.6 `pedagogical_lead` — Responsable pédagogique école

Nom technique : `"pedagogical_lead"` (`models.py:47`) Description : Supervise la qualité pédagogique au sein d'une école spécifique. Ne peut pas approuver pour B2B.

Fonctionnalités autorisées :

Fonctionnalité	Type	Endpoints
Rapport de progression école	Lecture	<code>GET /api/pedagogical-lead/progress-report</code> (<code>pedagogical_lead.py:62</code>)
Review local cours	Écriture	<code>PUT /api/pedagogical-lead/courses/{id}/review-local</code> (<code>pedagogical_lead.py:89</code>)
Escalader vers <code>pedagogical_admin</code>	Écriture	<code>POST /api/pedagogical-lead/escalate/{id}</code> (<code>pedagogical_lead.py:130</code>)
Réaffecter un prof à une classe	Écriture	<code>POST /api/pedagogical-lead/classes/{id}/reassign-teacher</code> (<code>pedagogical_lead.py:160</code>)
Reporter un devoir	Écriture	<code>PUT /api/pedagogical-lead/assignments/{id}/postpone</code> (<code>pedagogical_lead.py:197</code>)

Fonctionnalité	Type	Endpoints
Performance école	Lecture	<code>GET /api/pedagogical-lead/performance</code> (pedagogical_lead.py:221)
Analyse questions IA	Lecture	<code>GET /api/pedagogical-lead/ai-questions-analysis</code> (pedagogical_lead.py:249)
Tout ce qu'un admin_school fait	*	(via <code>require_admin</code>)

Fonctionnalités interdites (réservées pedagogical_admin) :
• Approuver pour B2B (<code>approved_for_b2b</code>) • Gérer les packs d'étude • Voir les escalades de toutes les écoles • Voir la couverture curriculum globale

Endpoints API accessibles :	7 endpoints pédagogiques + ~65 endpoints admin	Pages frontend : <code>/dashboard/school/*</code> (SchoolAdminLayout) avec accès exclusif à "Pédagogie"
-----------------------------	--	---

3. Matrice Rôles x Fonctionnalités

Fonctionnalité	student	teacher	admin_school	super_admin	pedagogical_admin	pedagogical_lead
<code>Catalogue & Contenu</code>						
Parcourir catalogue public	✓	✓	✓	✓	✓	✓
Voir détail cours public	✓	✓	✓	✓	✓	✓
Consulter ses cours inscrits	✓	✓	✓	✓	✓	✓
Suivre une leçon	✓	✓	✓	✓	✓	✓
Passer un quiz	✓	✓	✓	✓	✓	✓

Fonctionnalité	student	teacher	admin_school	super_admin	pedagogical_admin	pedagogical_lead
Télécharger certificat	✓	✓	✓	✓	✓	✓
Création de contenu						
Créer un cours	✗	✓	✓	✓	✓	✓
Modifier un cours (auteur)	✗	✓	✓	✓	✓	✓
Créer chapitres/leçons	✗	✓	✓	✓	✓	✓
Soumettre pour review	✗	✗	✓	✓	✓	✓
Review pédagogique						
Voir cours en attente	✗	✗	✗	✓	✓	✗
Approuver pour B2B	✗	✗	✗	✓	✓	✗
Approuver local (école)	✗	✗	✗	✓	✓	✓
Demander révision	✗	✗	✗	✓	✓	✓
Escalader vers plateforme	✗	✗	✗	✗	✗	✓
Gestion école						
Gérer utilisateurs école	✗	✗	✓	✓	✓	✓
Créer/supprimer classes	✗	✓	✓	✓	✓	✓
Gérer enrollments	✗	✗	✓	✓	✓	✓

Fonctionnalité	student	teacher	admin_school	super_admin	pedagogical_admin	pedagogical_lead
Approuver enseignant	✗	✗	✓	✓	✓	✓
Import CSV élèves	✗	✗	✓	✓	✓	✓
Gérer settings école	✗	✗	✓	✓	✓	✗
Voir finance école	✗	✗	✓	✓	✓	✗
Gestion plateforme						
Gérer toutes les écoles	✗	✗	✗	✓	✗	✗
Supprimer utilisateur	✗	✗	✗	✓	✗	✗
Supprimer école	✗	✗	✗	✓	✗	✗
Stats globales	✗	✗	✗	✓	✗	✗
Settings plateforme (API keys)	✗	✗	✗	✓	✗	✗
Audit logs global	✗	✗	✗	✓	✗	✗
Gérer vérifications identité	✗	✗	✗	✓	✗	✗
Rapport marge	✗	✗	✗	✓	✗	✗
Gérer packs d'étude	✗	✗	✗	✓	✓	✗
Couverture curriculum globale	✗	✗	✗	✓	✓	✗
IA & Portefeuille						

Fonctionnalité	student	teacher	admin_school	super_admin	pedagogical_admin	pedagogical_lead
Tuteur IA (ask/explain/quiz)	✓	✓	✓	✓	✓	✓
Ingestion documents IA	✗	✓	✓	✓	✓	✓
AI Factory (génération)	✗	✗	✓	✓	✓	✓
Consulter portefeuille	✓	✓	✓	✓	✓	✓
Acheter crédits	✓	✓	✓	✓	✓	✓
Allouer crédits (admin)	✗	✗	✗	✓	✗	✗
Communication						
Boîte réception	✓	✓	✓	✓	✓	✓
Envoyer broadcast	✗	✗	✓	✓	✓	✓
Packs & cours B2B						
Voir catalogue packs	✓	✓	✓	✓	✓	✓
Acheter pack (student)	✓	✗	✗	✗	✗	✗
Acheter pack école	✗	✗	✓	✓	✓	✓
Accorder cours B2B école	✗	✗	✗	✓	✗	✗

Légende : ☒ Autorisé | ☒ Interdit | ☐ Partiel (voir notes détaillées)

4. Règles d'authentification par rôle

4.1 Mécanisme d'authentification global

Paramètre	Valeur	Source
Mécanisme	JWT (Bearer token)	<code>auth.py:33</code> — <code>OAuth2PasswordBearer(tokenUrl="/auth/1</code>
Algorithme	HS256	<code>config.py:17</code> — <code>jwt_algorithm = "HS256"</code>
Clé de signature	<code>settings.jwt_secret</code>	<code>config.py:16</code> — min 32 chars en production
Durée de vie token	1440 minutes (24h)	<code>config.py:18</code> — <code>jwt_expire_minutes = 1440</code>
Refresh tokens	NON IMPLÉMENTÉ	Aucun mécanisme de refresh trouvé
Claims du token	<code>sub</code> (user_id), <code>school_id</code> , <code>exp</code>	<code>auth.py:25-28</code>

4.2 Hashing des mots de passe

Paramètre	Valeur	Source
Algorithme	bcrypt	<code>security.py:1</code> — <code>import bcrypt</code>
Salt	Auto-généré (<code>bcrypt.gensalt()</code>)	<code>security.py:10</code>
Rounds	Défaut bcrypt (10)	Non configuré explicitement

4.3 Règles de mot de passe

Implémentées dans `validation.py:82-105` :

- Minimum 8 caractères (ligne 91)
- Maximum 128 caractères (ligne 94)
- Au moins une lettre (ligne 98)
- Au moins un chiffre (ligne 102)
- `NON vérifiés` : majuscule, caractère spécial, mot de passe commun

4.4 Authentification par rôle — Pas de différenciation

Tous les rôles utilisent le même mécanisme d'authentification. Il n'y a pas de :

- 2FA / MFA
- Vérification IP/device
- Verrouillage de compte après tentatives échouées
- Rotation forcée du mot de passe
- Durée de vie différente du token par rôle

Différence unique : Les comptes désactivés (`is_active=False`) sont rejetés par `get_current_user` (`auth.py:55`) avec un HTTP 403, mais le endpoint `login` (`auth.py:211-226`) ne vérifie PAS `is_active` — un compte désactivé peut obtenir un token valide qui sera rejeté à l'appel suivant.

4.5 isolation multi-tenant

Mécanisme	Statut	Source
<code>school_id</code> sur User	✅ Implémenté	<code>models.py:280</code>
<code>check_school_access()</code>	✅ Implémenté	<code>deps.py:83-96</code>
Filtre DB automatique (TenantMixin)	❌ DÉSACTIVÉ	<code>db.py:70-72</code> — <code>TenantMixin = None</code>
<code>unique(school_id, email)</code>	✅ Contrainte DB	<code>models.py:272</code>

Risque : L'isolation multi-tenant repose ENTièrement sur les vérifications applicatives (`check_school_access`). Si un endpoint oublie cet appel, les données d'une école sont exposées à une autre.

5. Flux d'inscription et d'attribution des rôles

5.1 Inscription élève

Étape	Détail	Source
Route	<code>POST /auth/register</code>	<code>auth.py:64-109</code>
Rôle attribué	<code>"student"</code> (forcé côté serveur, ligne 83)	<code>auth.py:83</code>
Validation	Email unique + password strength	<code>auth.py:67-86</code>

Étape	Détail	Source
Crédits	100 TRIAL (expire 30j)	<code>auth.py:102-105</code>
Approbation	<code>Aucune</code> — accès immédiat	—
UI	<code>RegisterPage.tsx</code> — sélecteur rôle + niveau scolaire	<code>RegisterPage.tsx</code>

5.2 Inscription enseignant (essai démo)

Étape	Détail	Source
Route	<code>POST /auth/register-trial-teacher</code>	<code>auth.py:</code>
Rôle attribué	<code>TEACHER</code> (forcé côté serveur, ligne 141)	<code>auth.py:</code>
Champs forcés	<code>school_id=demo_school</code> , <code>subscription_plan=TRIAL</code> , <code>is_demo_account=True</code>	<code>auth.py:</code>
Crédits	100 TRIAL (expire 30j)	<code>auth.py:</code>
Approbation	<code>Aucune</code> — accès immédiat	—
UI	<code>RegisterPage.tsx</code> — bouton "Commencer l'essai gratuit"	<code>Register</code>

5.3 Inscription enseignant (établissement réel)

Étape	Détail	Source
Route	<code>POST /auth/teacher-register</code>	<code>auth.py:171-208</code>
Résultat	Crée un <code>TeacherRegistration</code> (status=PENDING), <code>PAS</code> de <code>compte utilisateur</code>	<code>auth.py:199-204</code>
Approbation requise	<code>Oui</code> — <code>admin_school</code> doit approuver	<code>admin.py:1779</code>
Après approbation	Le compte utilisateur est créé (logique dans <code>admin.py:1779+</code>)	—
UI	<code>RegisterPage.tsx</code> → soumission → page de confirmation	<code>RegisterPage.tsx</code>

5.4 Attribution des autres rôles

Rôle	Qui l'attribue	Mécanisme	Source
<code>admin_school</code>	<code>super_admin</code>	<code>PUT /api/admin/users/{id}</code> (champ <code>role</code>)	<code>admin.py:494</code>
<code>pedagogical_admin</code>	<code>super_admin</code>	<code>PUT /api/admin/users/{id}</code> (champ <code>role</code>)	<code>admin.py:494</code>
<code>pedagogical_lead</code>	<code>super_admin</code>	<code>PUT /api/admin/users/{id}</code> (champ <code>role</code>)	<code>admin.py:494</code>
<code>teacher</code> (<code>school_affiliated</code>)	<code>admin_school</code>	<code>POST /api/admin/teacher-registrations/{id}/review</code>	<code>admin.py:1779</code>

Restriction : `admin_school` peut assigner les rôles `student`, `teacher`, `pedagogical_admin`, `pedagogical_lead` mais PAS `super_admin` (`admin.py:605-618`).

6. Failles et incohérences détectées

CRITIQUE

#	Finding	Fichier:Ligne	Impact
C1	<code>TenantMixin = None</code> — le filtre DB automatique multi-tenant est désactivé. L'isolation des données entre écoles repose uniquement sur les appels manuels à <code>check_school_access()</code> .	<code>db.py:70-72</code>	Toute donnée d'une école peut fuiter vers une autre si un endpoint oublie <code>check_school_access</code>
C2	Routes <code>/dashboard/*</code> (teacher/student) sans <code>RequireRole</code> — tout utilisateur authentifié (même student) peut accéder aux routes teacher (<code>/dashboard/teacher/classroom</code> , etc.)	<code>App.tsx:158-189</code>	Un élève peut accéder à l'interface enseignant côté frontend
C3	<code>submit_for_review</code> ne vérifie pas l'état initial — le endpoint <code>POST /{id}/submit-for-review</code> définit <code>pedagogical_status = "pending_review"</code> directement	<code>admin_courses.py:369-383</code>	Un cours déjà publié peut être remis en "pending_review"

#	Finding	Fichier:Ligne	Impact
	sans appeler <code>can_transition_status()</code>		

MOYEN

#	Finding	Fichier:Ligne	Impact
M1	Login ne vérifie pas <code>is_active</code> — un compte désactivé reçoit un token valide	<code>auth.py:211-226</code>	Faux sentiment de sécurité désactivation est repoussée au prochain appel API
M2	<code>require_non_demo_access</code> est un no-op — la fonction retourne toujours l'utilisateur sans rien bloquer	<code>deps.py:181-194</code>	Les comptes démo peuvent accéder à des ressources d'écoles si <code>check_school_access</code> n'est pas appelé en aval
M3	Sessions DB créées dans <code>check_course_ownership</code> — <code>SessionLocal()</code> au lieu de réutiliser la session de la requête	<code>deps.py:139-153, 158-173</code>	Risque d'épuisement du pool de connexions sous charge
M4	<code>admin_school</code> dans <code>require_admin</code> — les <code>admin_school</code> peuvent accéder à des endpoints réservés aux admins plateforme si le endpoint utilise uniquement <code>require_admin</code>	<code>deps.py:22-27</code>	Un <code>admin_school</code> peut théoriquement accéder à des settings ou opérations qui devraient concerner que des écoles
M5	Routes <code>/learn/*</code> sans authentification — les routes publiques <code>/learn/courses/*</code> n'ont pas de <code>RequireAuth</code>	<code>App.tsx:192-194</code>	Nécessaire pour le catalogue public, mais doit être documenté comme intentionnel
M6	Audit logs endpoint (<code>admin.py:2022</code>) ne filtre pas par école pour les non-super_admin	<code>admin.py:2022</code>	Un <code>admin_school</code> peut voir les audit logs d'autres administrateurs
M7	<code>enrollment_trends</code> et <code>api_cost_trends</code> ne filtrent pas par école pour les non-super_admin	<code>admin.py:1665, 1702</code>	Fuite de données cross-école dans les analytics

FAIBLE

#	Finding	Fichier:Ligne
F1	Pas de refresh token — le token expire après 24h sans mécanisme de renouvellement	<code>config.py:18</code>

#	Finding	Fichier:Ligne
F2	Pas de 2FA	—
F3	Pas de rate limiting sur le login	auth.py:211
F4	Pas de verrouillage de compte après tentatives échouées	auth.py:211-226
F5	AdminCoursesPage sans différenciation frontend — tous les boutons (supprimer, publier) visibles quel que soit le rôle	AdminCoursesPage.tsx:240-283
F6	Case sensitivity incohérente — SchoolAdminLayout.tsx:21 utilise === "pedagogical_lead" (lowercase) tandis que App.tsx:67 utilise .toUpperCase()	SchoolAdminLayout.tsx:21
F7	JWT secret fallback en dev — si la variable d'environnement n'est pas définie, un secret connu est utilisé	config.py:63
F8	Pas de rotation des mots de passe — les mots de passe seed (password123, DemoStudent2024!) ne sont jamais forcés à être changés	seed_full.py:92, seed_demo_school.py
F9	Le teacher-register valide le mot de passe même si aucun compte n'est créé (le mot de passe est jeté)	auth.py:182-184

7. Recommandations d'amélioration

Priorité CRITIQUE

#	Recommandation	Effort
R1	Activer le filtre multi-tenant DB — Implémenter le <code>TenantMixin</code> ou au minimum ajouter un test automatisé qui vérifie que chaque endpoint avec accès DB utilise <code>check_school_access</code>	Moyen
R2	Ajouter <code>RequireRole</code> aux routes <code>/dashboard</code> teacher/student — Restreindre les routes <code>/dashboard/teacher/*</code> aux rôles <code>teacher</code> et <code>admin_school</code> , et <code>/dashboard/courses</code> , <code>/dashboard/wallet</code> aux rôles <code>student</code> et <code>teacher</code>	Faible
R3	Corriger <code>submit_for_review</code> — Appeler <code>can_transition_status(db, course, "pending_review", current_user)</code> au lieu de définir le statut directement	Faible

Priorité MOYENNE

#	Recommandation	Effort
R4	Vérifier <code>is_active</code> dans le login — Ajouter <code>if not user.is_active: raise HTTPException(403)</code> avant de créer le token	Faible
R5	Rate limiting sur <code>/auth/login</code> — Utiliser <code>slowapi</code> ou un middleware custom (ex: 5 tentatives / 15 min par IP)	Moyen
R6	Renommer <code>require_admin</code> en <code>require_any_admin</code> et créer <code>require_school_admin</code> pour les endpoints qui ne devraient concerner qu'un admin_school	Faible
R7	Corriger les endpoints analytics sans filtrage école — Ajouter <code>check_school_access</code> dans <code>enrollment_trends</code> , <code>api_cost_trends</code> , et <code>list_audit_logs</code> pour les non-super_admin	Faible
R8	Fermer la session DB dans <code>check_course_ownership</code> — Utiliser la session existante ou passer <code>db</code> en paramètre	Moyen
R9	Ajouter du filtre frontend dans <code>AdminCoursesPage</code> — Masquer les boutons "Supprimer" et "Publier" pour les rôles qui n'ont pas ces permissions	Moyen

Priorité FAIBLE

#	Recommandation	Effort
R10	Implémenter un refresh token — Ou au minimum prolonger la durée de vie du token et ajouter un mécanisme de renouvellement silencieux	Élevé
R11	Ajouter 2FA pour les rôles super_admin et pedagogical_admin — Via TOTP (Google Authenticator)	Élevé
R12	Verrouillage de compte — Après 5 tentatives échouées, bloquer 15 minutes	Moyen

#	Recommandation	Effort
R13	Renforcer les règles de mot de passe — Exiger majuscule + minuscule + caractère spécial	Faible
R14	Forcer le changement de mot de passe pour les comptes seed/demo lors de la première connexion	Faible
R15	Documenter explicitement les endpoints publics — Ajouter des annotations <code># Public endpoint</code> dans le code pour les endpoints intentionally unauthenticated	Faible

Fin du rapport.	Toutes les affirmations sont vérifiables aux fichiers et lignes indiqués. Aucune supposition n'a été faite pour les informations non trouvées dans le code analysé. Le rapport complet est ci-dessus. En résumé, les 3 failles critiques à corriger en priorité :
	<code>TenantMixin = None</code> (<code>db.py:72</code>) — l'isolation DB multi-tenant est désactivée - Routes teacher/student sans <code>RequireRole</code> (<code>App.tsx:158-189</code>) — tout utilisateur authentifié peut accéder à l'interface enseignant <code>submit_for_review</code> bypass le lifecycle (<code>admin_courses.py:369-383</code>) — ne vérifie pas l'état initial du cours
Voulez-vous que je corrige ces 3 failles critiques maintenant ?	